Model integration guide (MIG)

1. Introduction
2. Preparing the programs and setting up the development environment
    2.1 Git (Github desktop recommended)
        2.1.1 What is Git?
        2.1.2 Setting up Github Desktop
    2.2 Intellij
        2.2.1 Setting up
        2.2.2 Run configurations
3. How does the Gravelmon repository work, and where can I make changes?
4. Getting the necessary files from the Trello board
    4.1 Getting the files from a .bbmodel file
    4.2 Final contents of the folder
5. Actually integrating the model
    5.1 Moving our files into the Gravelmon repository
        5.1.1 Textures
        5.1.2 Animations and Model
    5.2 Changing the species file
        5.2.1 Special model features
    5.3 Adding our animations to the poser file
        5.3.1 Common Pitfalls
    5.4 Setting the scale
    5.5 Setting the correct Portrait and Summary positions
6. Creating a pull-request for the changes
7. Tips and Tricks

1. Introduction

In this document I'll be detailing how to integrate a model into the mod. This includes setting up the necessary programs and files, editing the species file, editing the poser file, placing the files in the correct folders, editing the Fakemon's data and finally creating a pull-request for your changes.
In this example, I will be editing Nidorook from Pokemon Infinity.

This guide assumes you are familiar with the json file type.

2. Preparing the programs and setting up the development environment

In this chapter we will be getting everything ready for us to start the integration process. The following instructions should only have to be followed once. If you already have either Github or Intellij setup for Gravelmon, feel free to skip the associated steps.

2.1 Git (Github desktop recommended)

At the end of this chapter, you should have a local version of the Gravelmon repository on your pc. This is required to proceed to the next step, *setting up Intellij*.

2.1.1 What is Git?

Git is a version control system (VCS) that allows developers to work together by synchronising their work onto a single ‘repository’, and tracking who changed what. A repository is a collection of files, usually a project or application, that can be accessed using git. If you are new to git, it’s recommended that you get yourself familiar with the following terms:

Branch
A version of the repository that diverges from the main working project. Branches can be a new version of a repository, experimental changes, or personal forks of a repository for users to alter and test changes.

Clone
A clone is a copy of a repository or the action of copying a repository. When cloning a repository into another branch, the new branch becomes a remote-tracking branch that can talk upstream to its origin branch (via pushes, pulls, and fetches).

Fetch
By performing a Git fetch, you are downloading and copying that branch's files to your workstation. Multiple branches can be fetched at once, and you can rename the branches when running the command to suit your needs.

Master
The primary branch of all repositories. All committed and accepted changes should be on the master branch. You can work directly from the master branch, or create other branches.

Merge
Taking the changes from one branch and adding them into another (traditionally master) branch. These commits are usually first requested via pull request before being merged by a project maintainer.

Pull/Pull Request
If someone has changed code on a separate branch of a project and wants it to be reviewed to add to the master branch, that someone can put in a pull request. Pull requests ask the repo maintainers to review the commits made, and then, if acceptable, merge the changes upstream. A pull happens when adding the changes to the master branch.

Push
Updates a remote branch with the commits made to the current branch. You are literally "pushing" your changes onto the remote.

Repository ("Repo")
In many ways, you can think of a Git repository as a directory that stores all the files, folders, and content needed for your project. What it actually is, is the object database of the project, storing everything from the files themselves, to the versions of those files, commits, deletions, et cetera. Repositories are not limited by user, and can be shared and copied (see: fork).

Stash
While working with Git, you may need to make multiple changes to files, but you may not want all changes to go in one commit. If you want to pause the changes you are working on now in favor of working on another issue or improvement, you can "stash" your changes, essentially clearing them from the staging area until the changes are called again. You can only stash one set of changes at a time.

You don't have to know them by hard, but it's recommended that you are familiar enough with them to know what they mean when they come up in the guide. If you are having trouble remembering, you can refer to this chapter as a cheatsheet.

2.1.2 Setting up Github Desktop

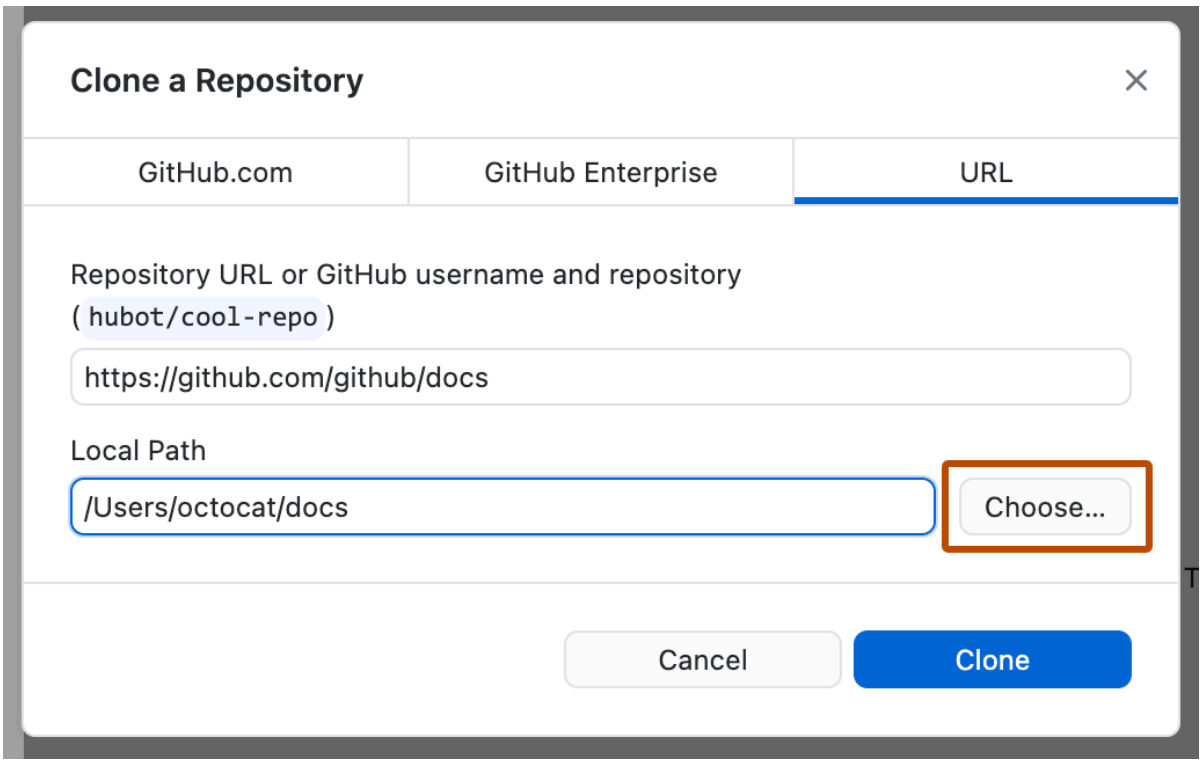
First, if you don't have a github account already, follow the guide to create an account:
https://docs.github.com/en/get-started/start-your-journey/creating-an-account-on-github

Next up, download, install, and set up Github Desktop. If you'd like to follow the cloning guide alongside this step, feel free to do so. Otherwise this guide will cover how to clone gravelmon in the next part.
https://docs.github.com/en/desktop/overview/getting-started-with-github-desktop

Now that we're all setup with Github Desktop, it's time to start using it. Follow the guide below to clone the Gravelmon repository:
https://docs.github.com/en/desktop/adding-and-cloning-repositories/cloning-a-repository-from-github-to-github-desktop

The Gravelmon repository:
https://github.com/StijnArts/Gravelmon

If you are having trouble with the last step, you can also clone the repository using the url. Follow this guide until it asks you to select a repository. Instead of using the 'GitHub.com' tab, use the 'URL' tab and fill in the Gravelmon repository URL:
https://docs.github.com/en/desktop/adding-and-cloning-repositories/cloning-and-forking-repositories-from-github-desktop



2.2 Intellij

Intellij is a program that allows us to edit Gravelmon files (and other Java projects). I recommend it for this guide.

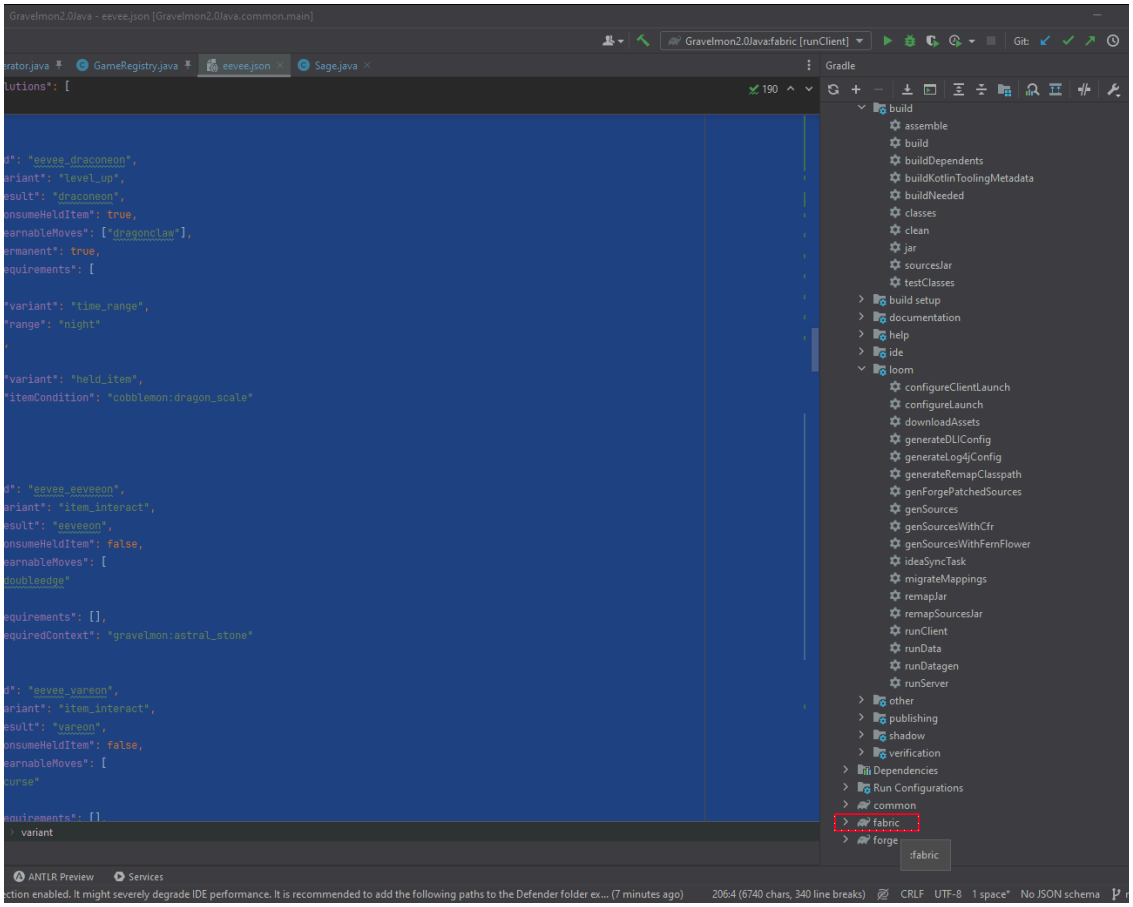
2.2.1 Setting up

To start with, download and install Intellij Community Edition (Do not download the ultimate edition):
https://www.jetbrains.com/products/compare/?product=idea&product=idea-ce

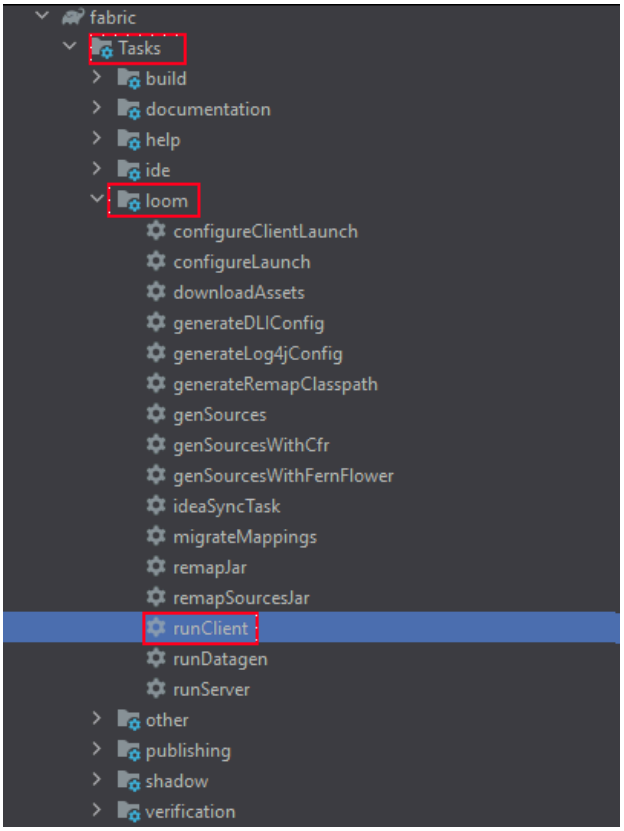
Then, once it's installed, open intellij and click 'Open' and navigate to where you've saved the Gravelmon repository on your local disk. Once there, click on the 'build.gradle' file in the Gravelmon2.0Java folder, and open it as a project. Intellij will now start setting up your development environment. This can take a while, so while it's busy doing that, feel free to read the docs about the UI of the program. The most important part for non-developers is how to navigate the file structure:
https://www.jetbrains.com/help/idea/guided-tour-around-the-user-interface.html

2.2.2 Run configurations

Once it's set up, navigate to your 'gradle' tab, and unfold the fabric category by clicking on it.



Then, navigate to the ‘runClient’ task to run the mod through the development environment, and double click on it. If you run into any issues here, contact Drai on discord. Take note of the ‘runDataGen’ task below the runClient task, as this is the task we need to run if we want our changes to the Pokemon data to be translated to usable game-files.



If all went well, you should be able to head in-game and play with the gravelmon features.

3. How does the Gravelmon repository work, and where can I make changes?

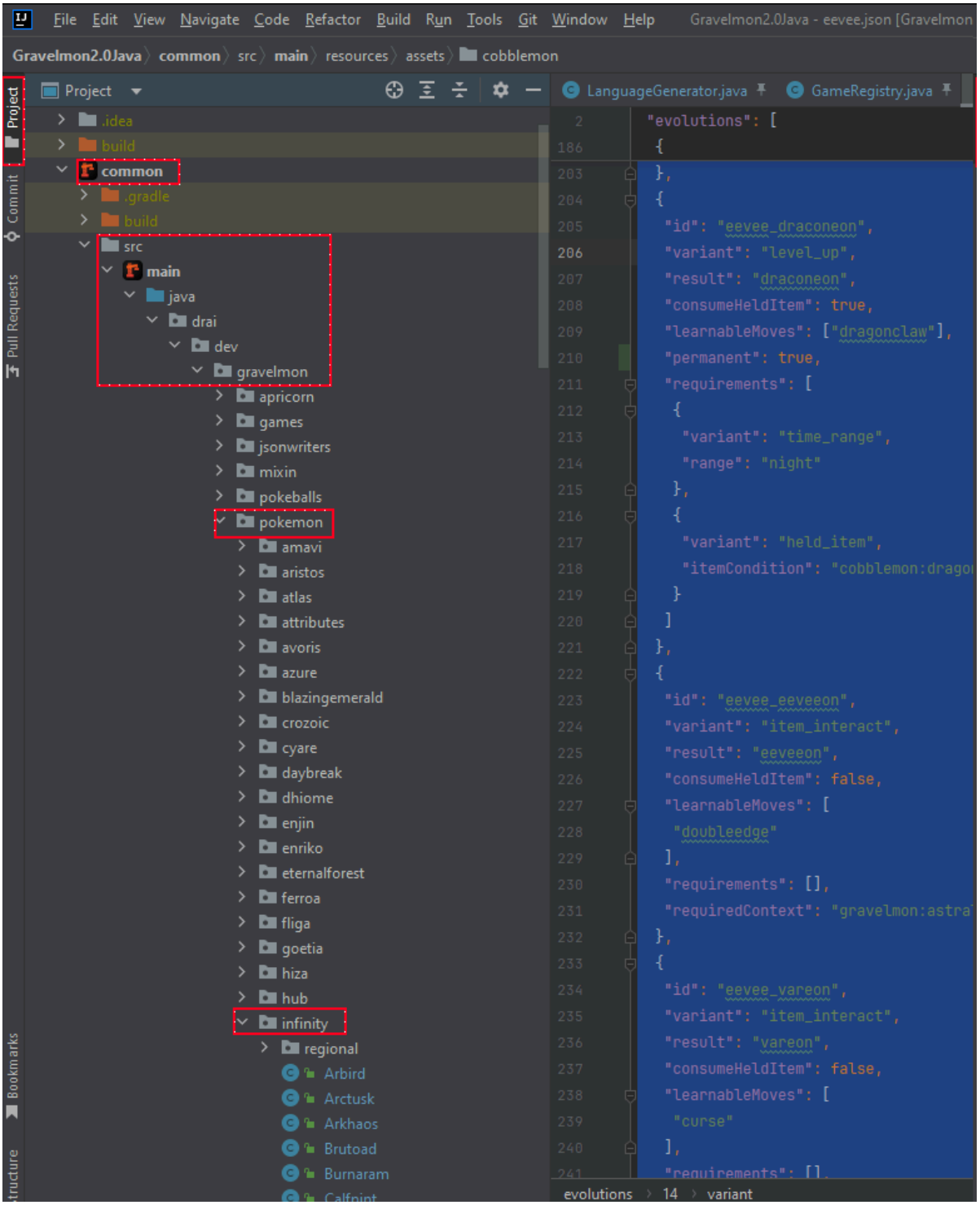
In this chapter I will be going over the workings of the Gravelmon data-generation system, for the part that's relevant to anyone reading this integration guide. Gravelmon uses data generation to create json assets. This means that in most cases, editing json files directly is pretty much useless since they will be overridden again when the data generator is run (which happens pretty often). As of right now, the only category of files that can be edited without issue directly are:

- data > gravelmon > recipes
- data > cobblemon > spawn_detail_presets
- data > cobblemon > species_additions
- data > cobblemon > tags
- assets > gravelmon

If you're familiar with the cobblemon model system, you'll have noticed that the 'assets > cobblemon > bedrock' folder wasn't in the list. This is because those assets are generated using code, for the most part. To start integrating a model, it's important that we tell the code that the pokemon for which we are adding a model should not be getting any more automated json.

To do this, navigate to the 'common > src > main > java > drai > dev > gravelmon > pokemon' folder (or otherwise known as a package), and find the folder with the name of the fangame/region the pokemon you want to edit is from. Expand that category by clicking on it,

and look for it in the list. The list is alphabetical. If the pokemon you want to integrate is a regional form, it'll be located in the regional folder inside of the fangame/region folder. In this example, I will be editing nidorook from Pokemon Infinity.
Note: If you are unsure which region your pokemon is from, check the ticket on the trello board.



Once found, open the file by double clicking on it. This will show you the java code containing all of the data for the pokemon you want to edit.

```
package drai.dev.gravelmon.pokemon.infinity;

import ...

public class Nidorook extends Pokemon {
    public Nidorook() {
        super( name: "Nidorook",
                Type.POISON, Type.GROUND,
                new Stats( HP: 78, attack: 97, defence: 75, specialAttack: 85, specialDefence: 70, speed: 100),
                List.of(Ability.POISON_POINT, Ability.MERCILESS), Ability.SHEER_FORCE,
                height: 14, weight: 768,
                new Stats( HP: 0, attack: 2, defence: 0, specialAttack: 0, specialDefence: 0, speed: 2), catchRate: 45,
                maleRatio: 0.5,
                baseExperienceYield: 214, ExperienceGroup.MEDIUM_SLOW,
                baseFriendship: 70,
                eggCycles: 52, List.of(EggGroup.FIELD, EggGroup.MONSTER),
                List.of( e1: "When a Nidorino or Nidorina evolve into Nidorook, they maintain their quadruped form. This
                List.of(),
                List.of(
                        new MoveLearnSetEntry(Move.BARBED_TACKLE, level: 1),
                        new MoveLearnSetEntry(Move.LEER, level: 1),
                        new MoveLearnSetEntry(Move.PECK, level: 1),
                        new MoveLearnSetEntry(Move.FOCUS_ENERGY, level: 1),
                        new MoveLearnSetEntry(Move.DOUBLE_KICK, level: 1),
                        new MoveLearnSetEntry(Move.POISON_STING, level: 1),
                        new MoveLearnSetEntry(Move.FURY_SWIPES, level: 1),
                        new MoveLearnSetEntry(Move.HORN_ATTACK, level: 1),
                        new MoveLearnSetEntry(Move.HELPING_HAND, level: 1),
                        new MoveLearnSetEntry(Move.BITE, level: 23),
                        new MoveLearnSetEntry(Move.BODY_SLAM, level: 35),
                        new MoveLearnSetEntry(Move.TOXIC_SPIKES, level: 40),
                        new MoveLearnSetEntry(Move.FLATTER, level: 43),
                        new MoveLearnSetEntry(Move.EARTH_POWER, level: 45),
                        new MoveLearnSetEntry(Move.POISON_JAB, level: 48),
                        new MoveLearnSetEntry(Move.POISON_FANG, level: 55),
                        new MoveLearnSetEntry(Move.EARTHQUAKE, level: 58),
                        new MoveLearnSetEntry(Move.MEGAHORN, level: 62),
                        new MoveLearnSetEntry(Move.DRAGON_CLAW, condition: "tm"),
```

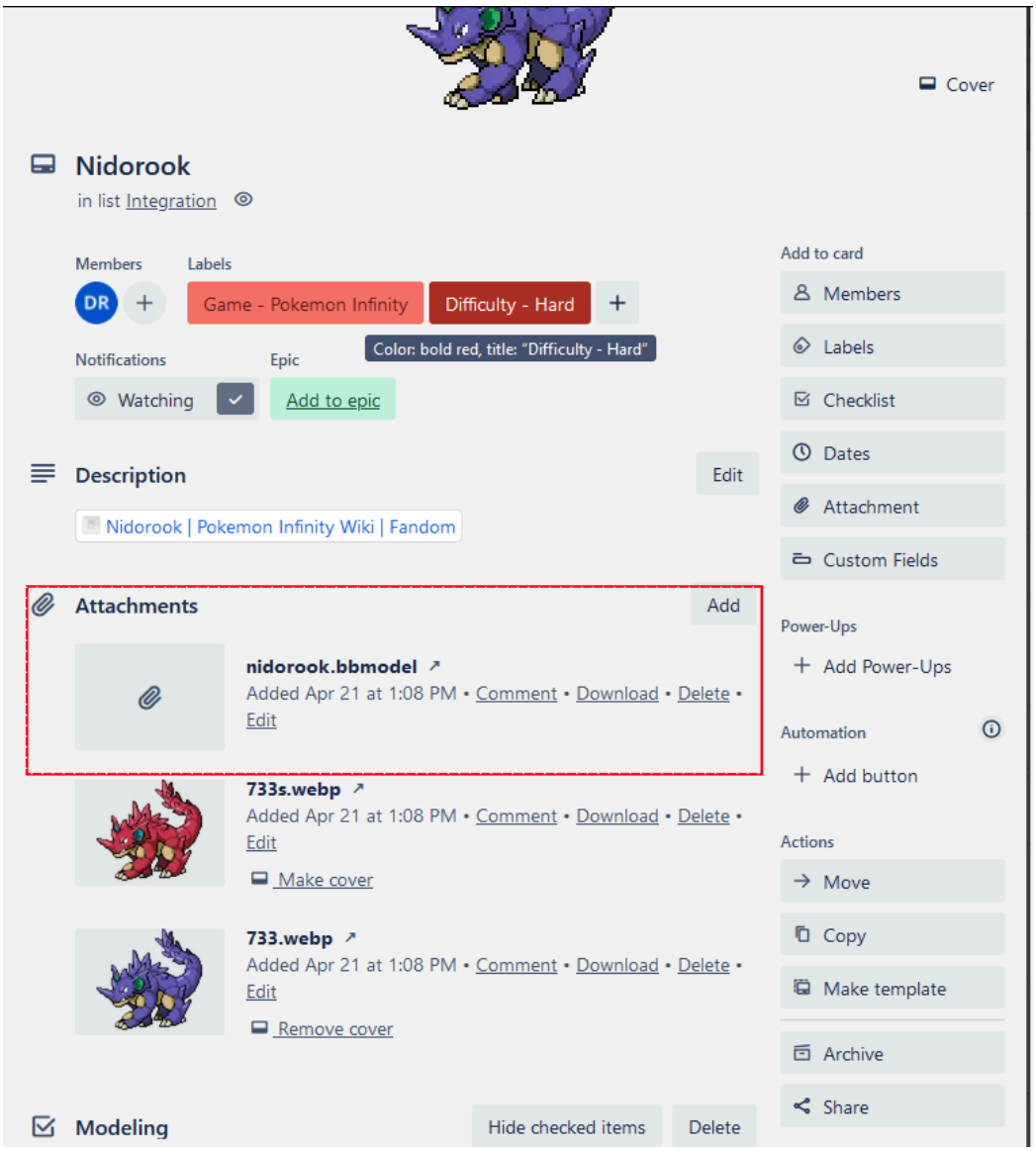
Next, scroll down to the bottom of the page, and add the following line: `setModeled(true);`
This should be done at the bottom of the page, above the last two curly brackets:

```
                        new SpawnCondition(SpawnConditionType.CANSEESKY, condition: "true")
                ), List.of(),
                List.of(SpawnPreset.NATURAL),
                baseScale: 0.32, portraitScale: 0.3,
                List.of());
        this.setPreEvolution("nidorino");
        setModeled(true);
    }
}
```

This line tells Gravelmon that we don't want it to generate any more asset files for us. Now we are free to edit the existing files.

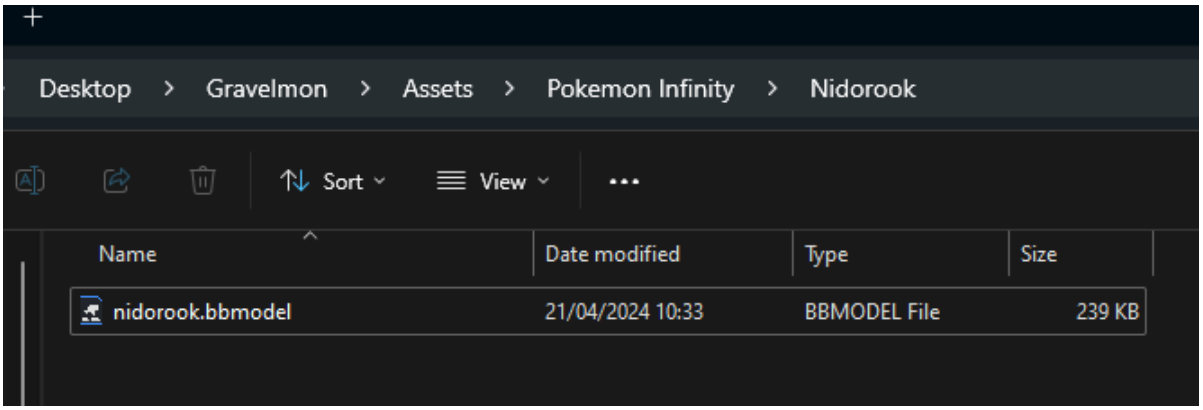
4. Getting the necessary files from the Trello board

The card you want to integrate should have attachments containing the necessary files for you to integrate the pokemon into gravelmon. This is usually either a zip file, or a bbmodel file. If it's the former, you're in luck. If it's the latter, follow the guide below to unpack the files into individual files for us to use during integration. Regardless of what type it is, create a new folder somewhere where you'll remember (not in the repository folder though), and download the file(s) to that folder.

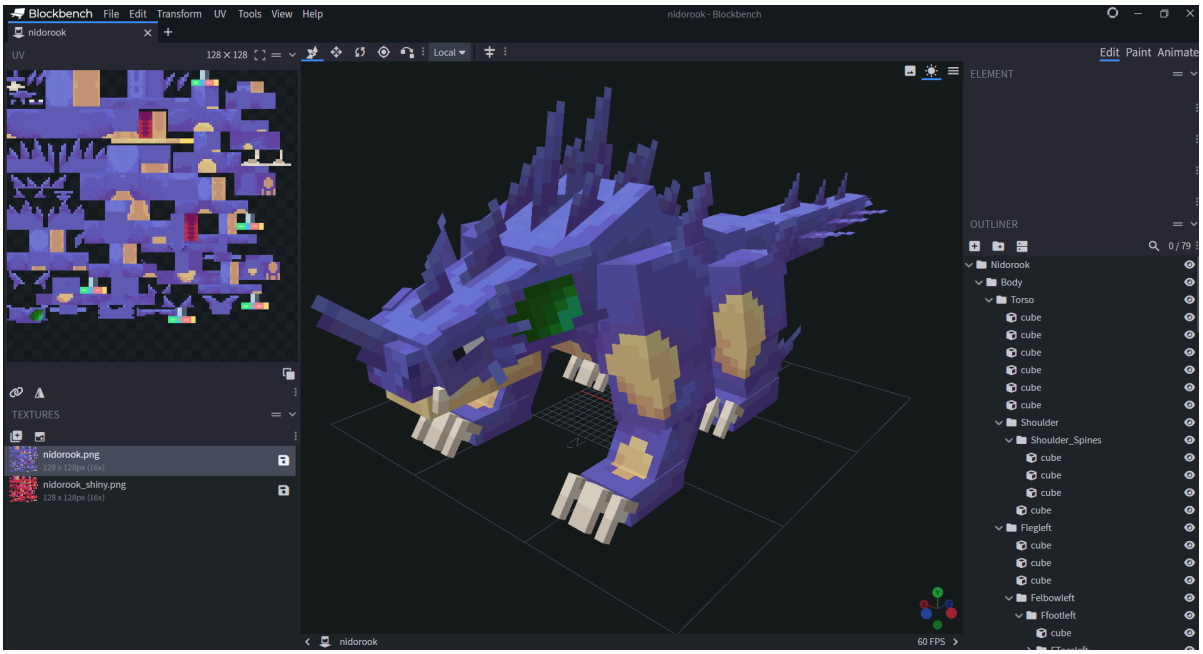


4.1 Getting the files from a .bbmodel file

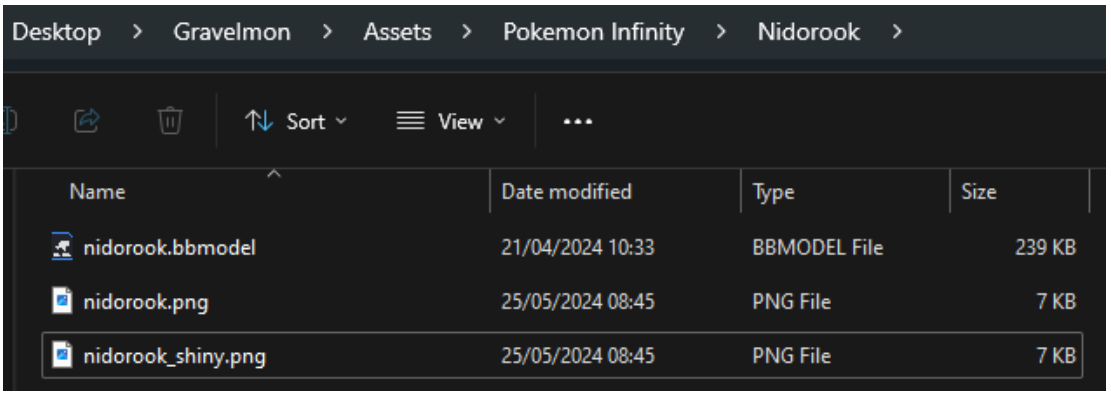
Assuming you've downloaded the file and put it in a new folder, you should have something that looks like this:



The next step is to open the file in BlockBench. If you've made it this far into the guide I'm going to assume you have blockbench installed. If not, quickly install it and come back to this guide. Opening the file should show you the model with the textures in a view similar to this:



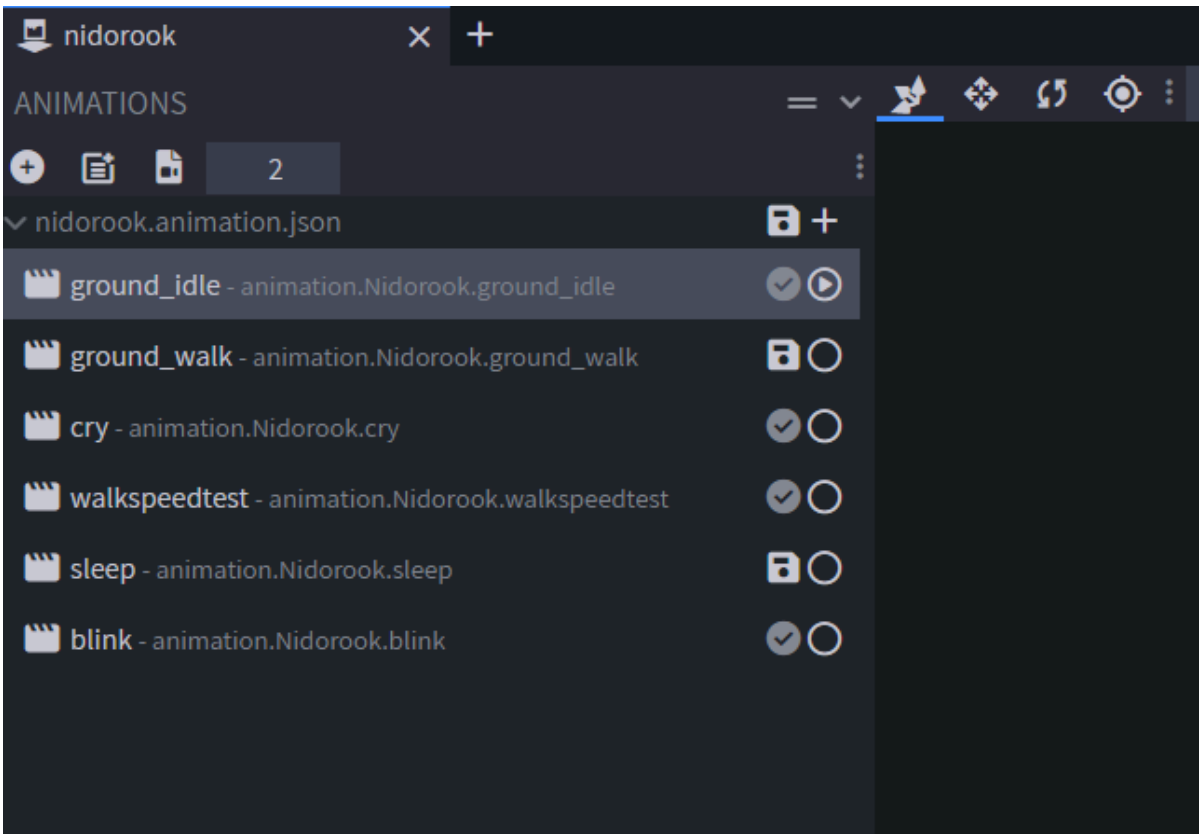
The first step is to save the textures. Click on the save icon next to the textures. When prompted for a location, open the folder containing the bbmodel file and save the textures to that folder.



Next, we navigate to the Animate tab, and check for the following things when inspecting the animations tab:

- Are the animation names and paths in lowercase?
- Does the animation path reference the correct pokemon?
- Are movement animations formatted as [movement type]_[state]? (ground_idle, ground_walk)

As an example of how to do this check, here's the animations for Nidorook as I've found them in the file.

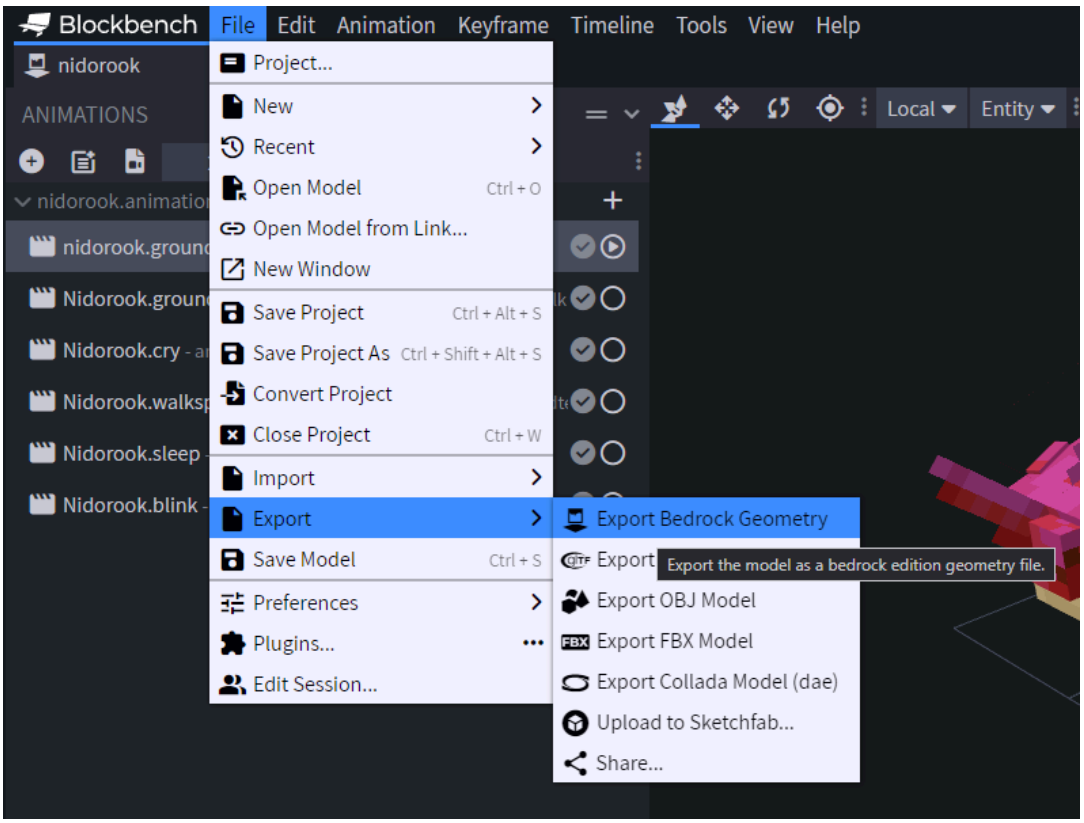


As you can see, all of the animations are named correctly, but the path is slightly off. Nidorook in the animation path (the grey text next to the name) is spelled with capitalization. Note: If your file has an error with the animation names being incorrectly formatted, it's recommended that you edit those inside of Blockbench by right-clicking and selecting rename before saving the file as a json file.

Now that we've found an error, we should remember what it was and proceed to save the animations to a json file. To do this, click the save icon next to the top fold-out menu (in the screenshot above that's 'nidorook.animation.json'). Save it to the folder we've created earlier. If your animations contained errors, we can fix them inside of the json file.

Navigate to the folder in which you saved the animations, and open the file. I will be using Notepad++ in the example, but feel free to use any tool you like. Since we found the spelling of Nidorook to be incorrect in the animation path, we can find and replace all occurrences of 'Nidorook' with 'nidorook'. Now our animation file is ready for use with cobblemon.

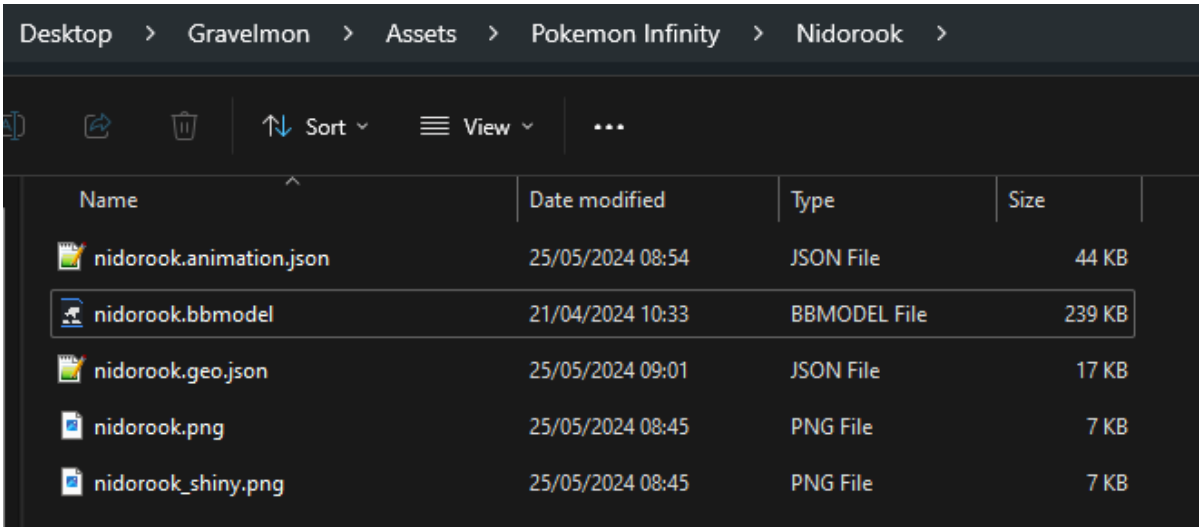
The last step is to save the geometry of the model. To do this, navigate to 'File > Export > Export Bedrock Geometry' and save the resulting .geo.json file to the folder.



If the file is not yet named with a prefix for the fangame/region the pokemon is from, rename the file. In our case, the file for the Nidorook model is currently named ‘nidorook.geo.json’, but it should be named ‘infinity_nidorook.geo.json’.

4.2 Final contents of the folder

Your folder should contain the animation file, the geo file, and the texture files for the model you will be integrating. It should look something like this:



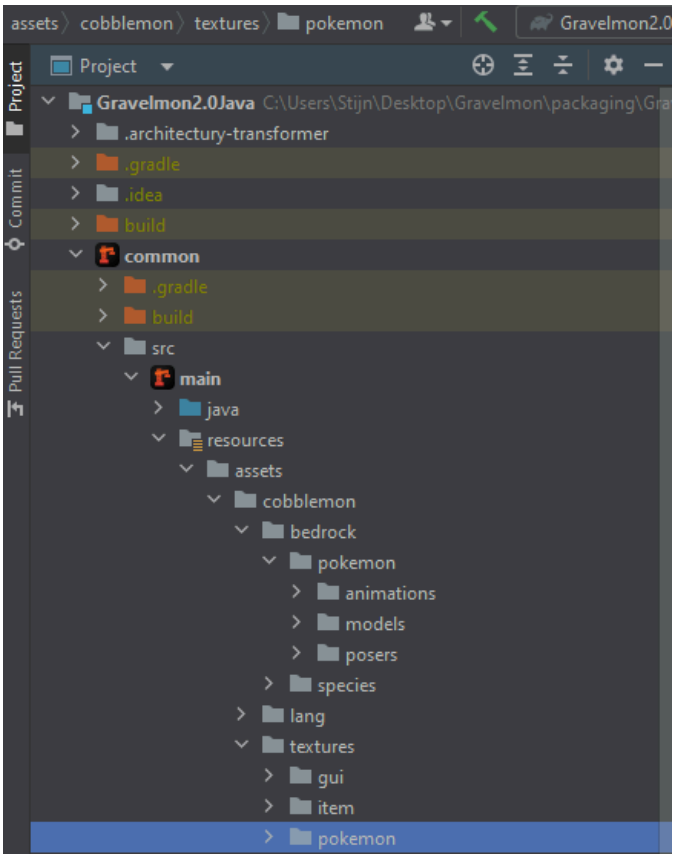
Note: if you’re working with a zip file, the .bbmodel file will be your zip file. If your zipfile contained a bbmodel file, refer to chapter 4.1 of this guide.

5. Actually integrating the model

Now that we have everything ready to integrate, we can finally start editing some json files. To start off with, let's move our model files to the correct folders inside of the Gravelmon repository.

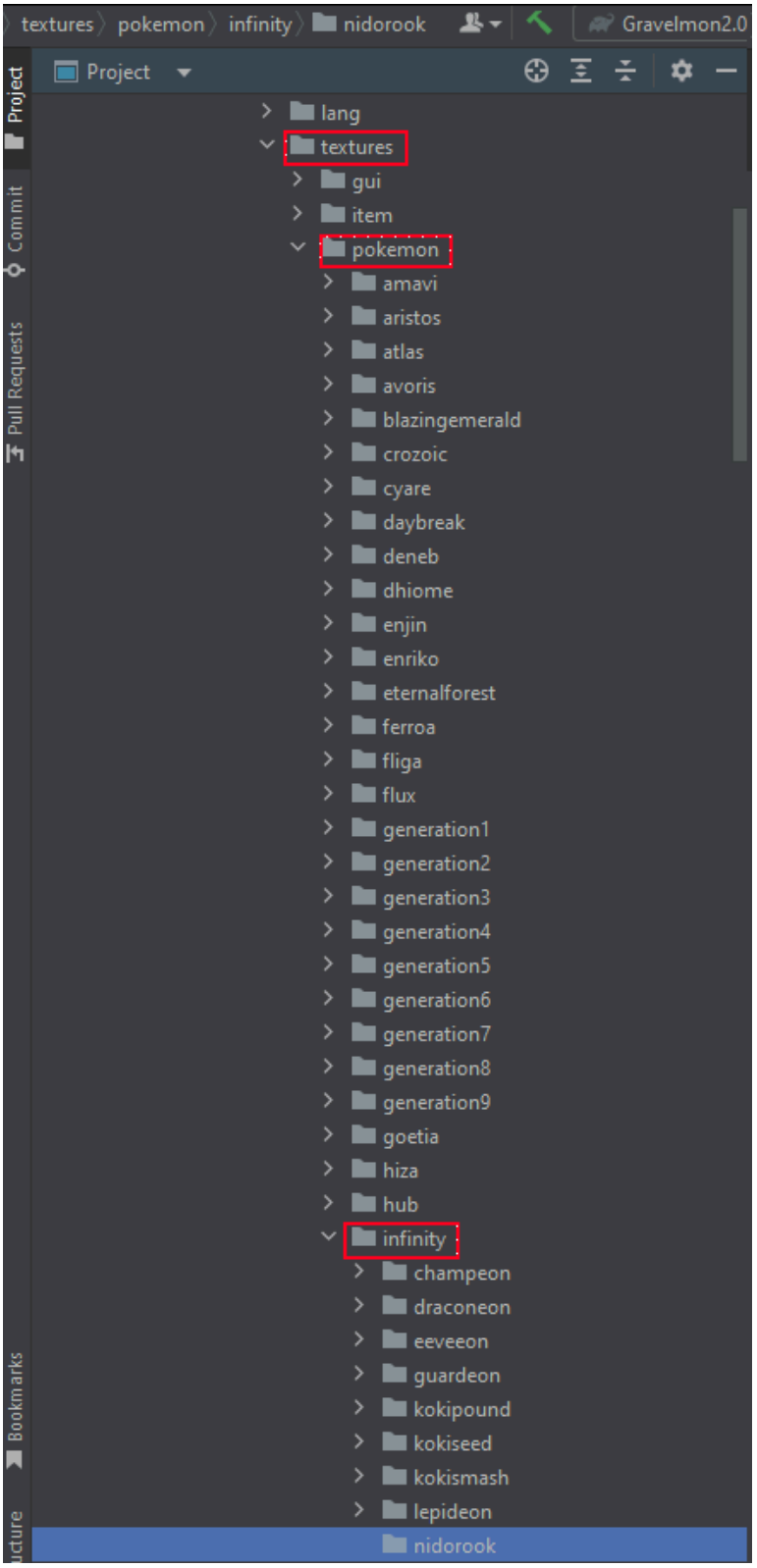
5.1 Moving our files into the Gravelmon repository

To start with, navigate to the cobblemon assets folder inside of the common package 'Gravelmon2.0Java > common > src > main > resources > assets > cobblemon'.

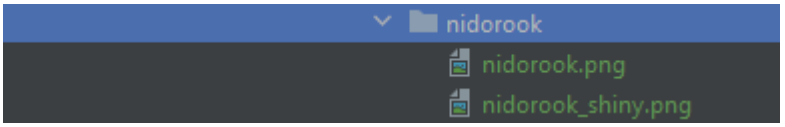


5.1.1 Textures

Find the 'textures>pokemon' folder (highlighted in the screenshot above), and open it. Navigate to the folder with the name of the fangame/region the Pokemon is from, and open it. In our case, it'll be the 'infinity' folder. Once in there, create a new folder with the name of the Pokemon we're currently integrating (in our case, we're creating the 'nidorook' folder).

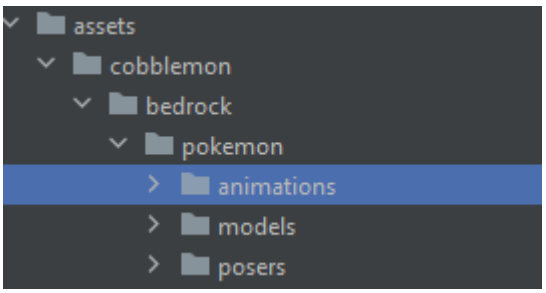


Now, **copy (not move)** the textures to the newly made folder. If git asks you to add the files, click yes (and check don't ask me again).



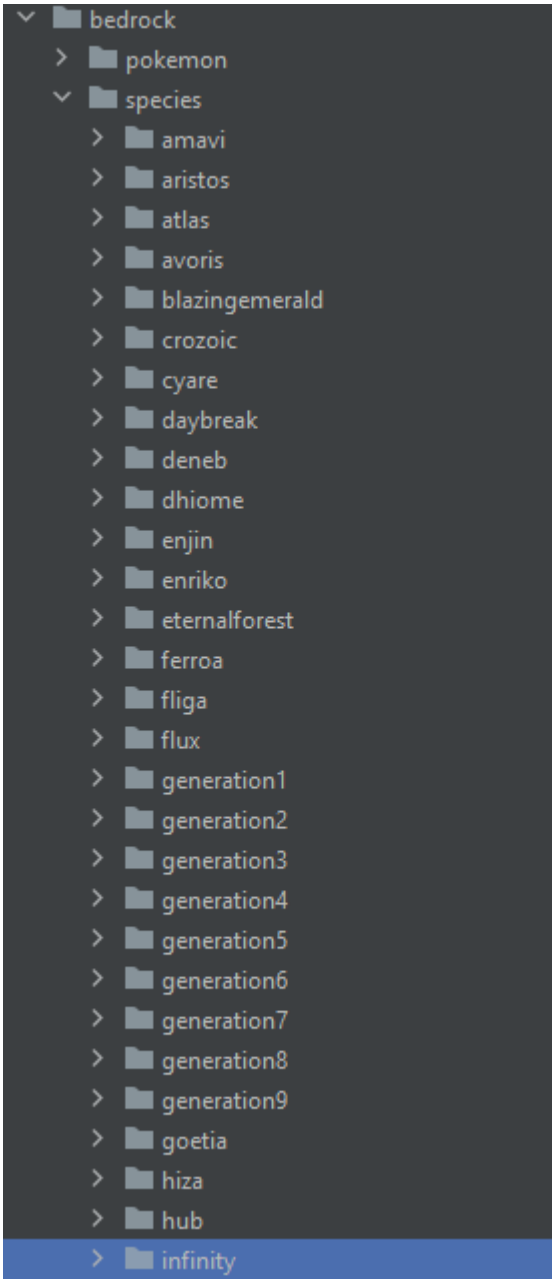
5.1.2 Animations and Model

Copy the animation file to the ‘bedrock > pokemon > animations’ folder. Then copy the model file to the ‘models’ folder that's beside the ‘animations’ folder.

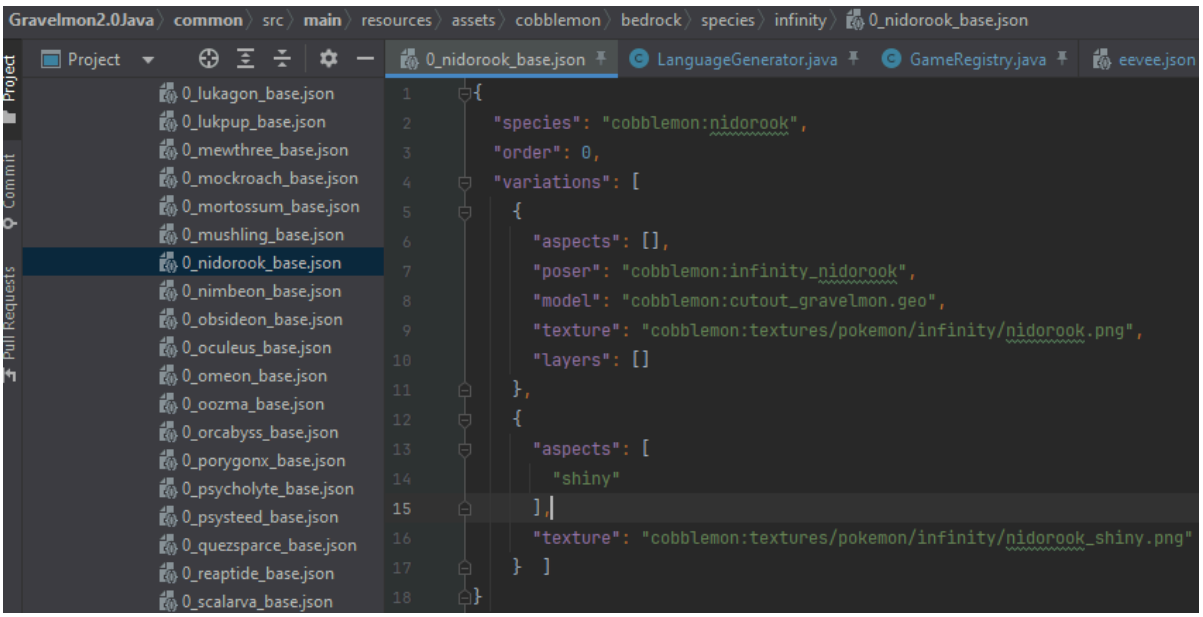


5.2 Changing the species file

Navigate to the bedrock folder from the previous step, and open the ‘species’ folder. There, find the folder with the name of the fangame/region your Pokemon is from.



Once there, find the file for your fakemon. The files are named in the pattern of ‘0_[fakemon name]_base.json’, so in our case it’ll be ‘0_nidorook_base.json’. Open the file and take a look at what’s inside.



Here we can see a couple things. First, there is a ‘variations’ array designating the different variations the pokemon can have. In our case, it’s just the shiny variation, but it’s also possible for this to be different gender forms for example. At the top of the ‘variations’ file is the first entry which details all of the settings that the variations below it will inherit from if not changed, since that variation’s ‘aspects' array is empty.
In this guide I will only be talking about the properties that are relevant for the integration process.
To point this file to the correct model, change the ‘model’ property’s value from ‘cobblemon:cutout_gravelmon.geo’ to the name of the model file we just placed in the ‘models’ folder. In our case, I will be renaming it to ‘cobblemon:infinity_nidorook.geo’.
Next, we’ll set the texture location. In most cases, all you’ll have to do is add an extra folder to the path. In the example, the current texture location is ‘cobblemon:textures/pokemon/infinity/nidorook.png’. I will be changing this to ‘cobblemon:textures/pokemon/infinity/**nidorook/**nidorook.png’. Note the bold part is the additional folder I’ve added to the path. This folder should be the same as the folder we created in step 5.1.1 to save our textures to. Do the same for the shiny variant. In most cases, this is all you need to do to get the model and textures in game.

To test if everything worked as expected, run the ‘runClient’ task from chapter 2.2.2 and check if you see any errors in the logs. Head into a single player creative world, and give yourself the pokemon you are integrating and throw it out to check if the models and textures are working correctly (for all of the variations). If something is broken, check the logs to see what might be wrong. If the textures are missing, check that the path you point to in the species file matches the location of the textures. If the model is missing, check for the same but the model this time. If you aren't able to solve the issue, send your logs in the cobblemon add-on-help channel and ask for assistance.
Note: If I you get an error along the lines of ‘Expected BEGIN_ARRAY but was BEGIN_OBJECT at line 35 column 15 path $.minecraft:geometry[0].bones[2].cubes[2].uv’, check who modelled the model and ask them to make sure that their model doesn’t use per-uv meshes.

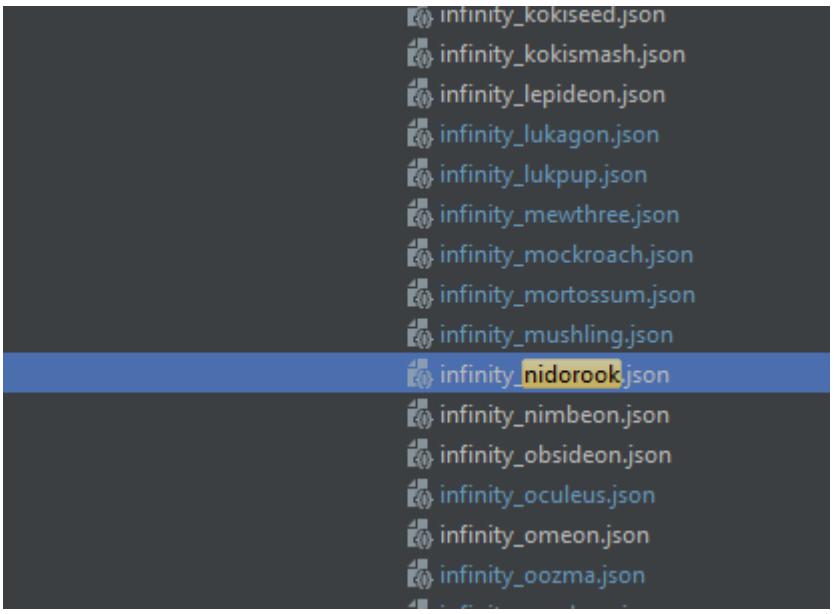
5.2.1 Special model features

This section will detail how to add animated textures and/or glowing textures to a model.

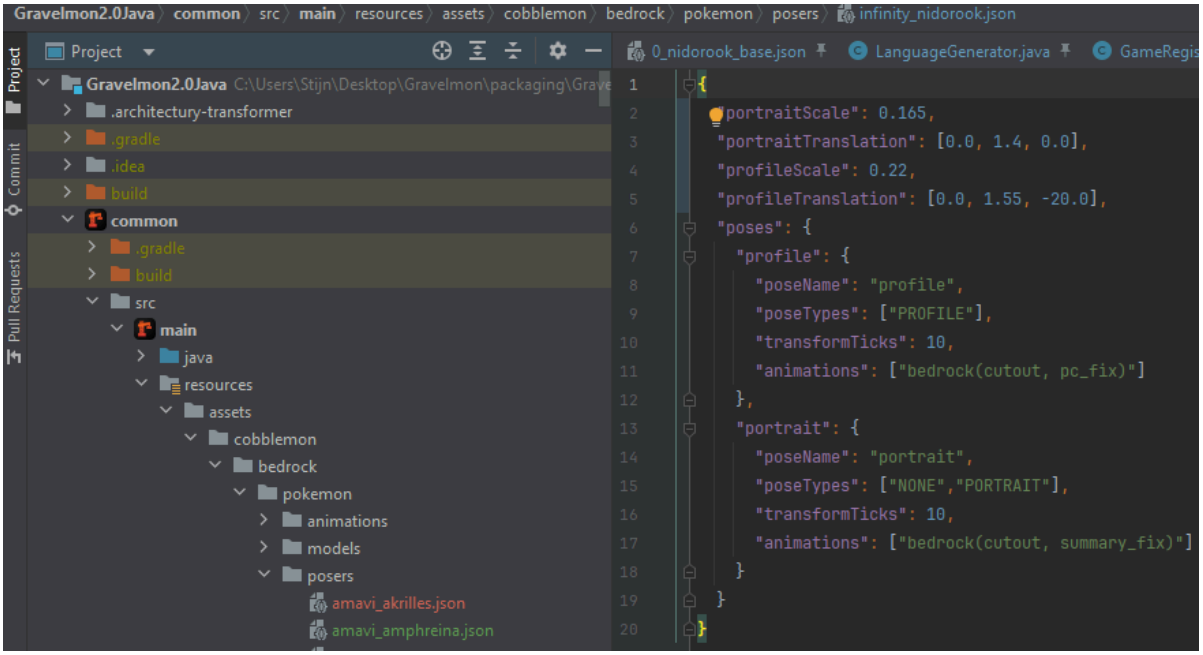
WIP COMING SOON If you need more info on this right away, check the

5.3 Adding our animations to the poser file

After we've confirmed that the models are working correctly, we can start configuring the poser files. Locate the poser file for the pokemon you are integrating, and open it. Poser files are located in the 'bedrock > pokemon > posers' folder, and have the same naming structure as models. This means I'll be looking for the 'infinity_nidorook.json' file.
TIP: with the poser folder expanded, you can type to search. For example, if I type 'nidorook' it'll find all files with that phrase in the name. It's highly recommended you do it this way, since there are well over 3000 poser files in that folder.



Open the file, and you should see something like this:



Now, open the animations file, or open the model in blockbench and navigate to the animations tab, and check which animations the model can use.
Since the process of adding the animations can be a bit complex, please refer to the official guide to get you started:https://wiki.cobblemon.com/index.php/Poser. The bottom of this page has an example that implements all the types of animations we generally feature, so use that for reference.
Tip: If you're interested in using the new poser structure, checkout this page instead. It's more complex than the old version, so it's recommended that you only do this once you know how the original version works: https://wiki.cobblemon.com/index.php/New_Poser

5.3.1 Common Pitfalls

- Crashing with missing bones: Usually this happens when the ‘head’ property is set to a bone that doesn’t exist in the model. This can happen if the model either doesn’t have a head bone (for example voltorb), or the head’s bone has a different name. Check if your model has either of these issues, and if not, ask for help in the cobblemon discord.

5.4 Setting the scale

Next up we will set the scale for the pokemon. Use the fangame wiki page for the pokemon for reference of how big it should be. If no size is listed, reference the size inside of the data-class we explored in chapter 3. Go into a singleplayer world, and use the /changesizeandscale command to play around with the correct scale and hitbox size for the mon. I recommend starting with 1 1 1, since most of the models are created to scale. Once you’ve found something you’re happy with, open up the data class file and add the following lines beneath the ‘setModeled’ line:

```
setBaseScale(1);
setHitbox(1,1);
```

Where ‘1’ is the correct value you’ve found during testing.

```
            ), SpawnContext.GROUNDED, SpawnPool.RARE, minSpawnLevel: 36, maxSpawnLevel: 51, sp
                    new BiomeSpawnCondition(List.of(Biome.IS_SAVANNA)),
                    new SpawnCondition(SpawnConditionType.TIMERANGE, condition: "day"),
                    new SpawnCondition(SpawnConditionType.CANSEESKY, condition: "true")
            ), List.of(),
            List.of(SpawnPreset.NATURAL),
            baseScale: 0.32, portraitScale: 0.3,
            List.of());
        this.setPreEvolution("nidorino");
        setModeled(true);
        setBaseScale(1);
        setHitbox( width: 1, height: 1);
    }
}
```

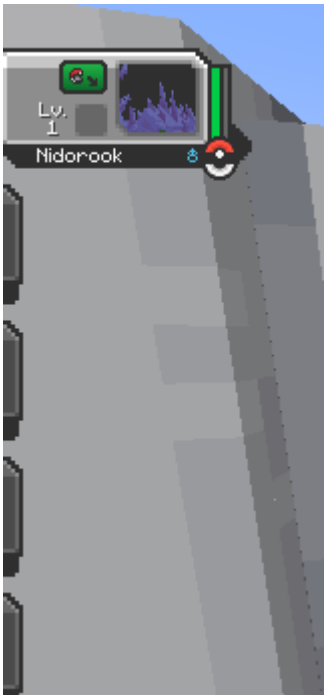
Now run the ‘rundatagen’ task that we saw in chapter 2.2.2. Once it’s finished, restart your game or build and do the /reload command to check if the values reflect what you expected the result to be.

Note: The current datagen will never stop the process on its own. Once it reads “All providers took: ..” you can safely close the process.

5.5 Setting the correct Portrait and Summary positions

Here we are, the final step. For this step it’s recommended that you place a pc in a single player world, and do /giveallpokemon in addition to giving yourself the pokemon you’re posing. This gives you some reference material to work off of. Sadly, the process involves a lot of trial and error. I recommend changing all of the translation values inside of the poser to 1,1,1, and the scale to 1 as well.

The ‘portrait’ values refer to the party ui when not looking at a cobblemon menu:

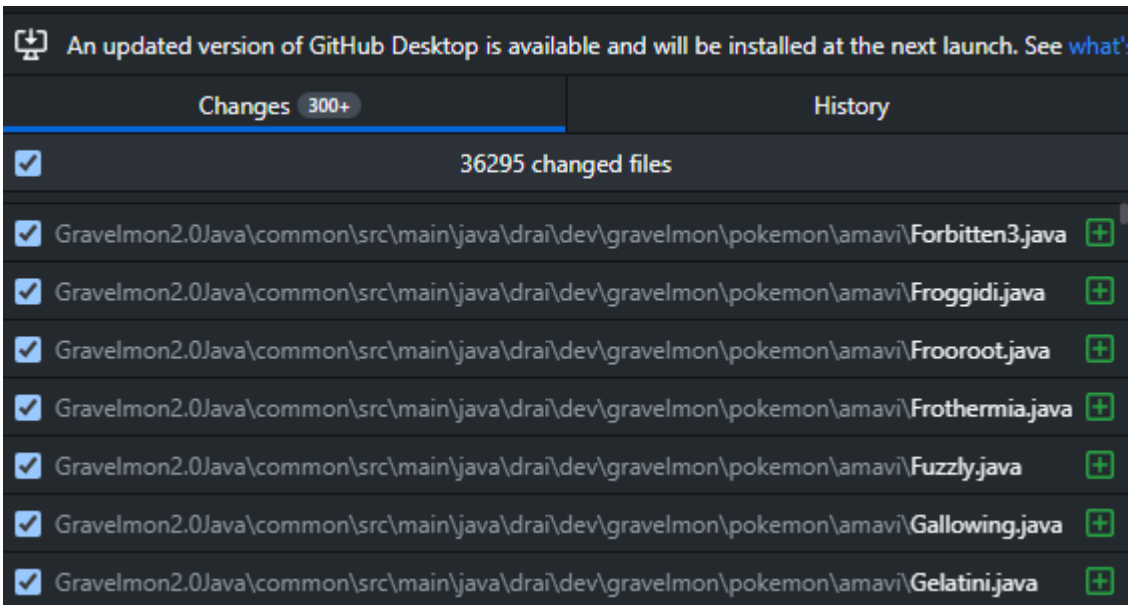


The ‘profile’ values refer to everything displayed inside of the pc and the summary window:

If the model looks too big to you in the corresponding view, lower the scale. If it looks too small, increase the scale. Same goes for the translation. This isn’t an exact science, but try to make it look similar to the cobblemon versions.

6. Creating a pull-request for the changes

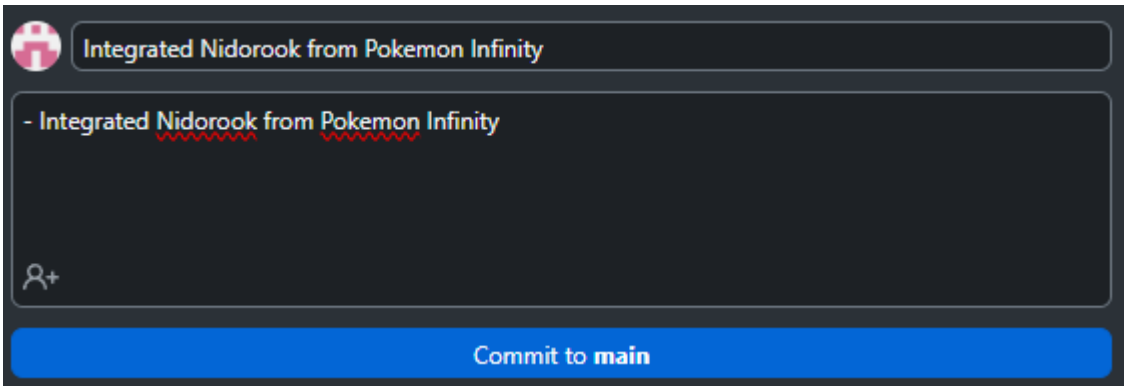
Congratulations, you’ve finished integrating your pokemon! Now there’s one last step in the process: creating a pull-request. If you’ve read the terms from chapter 2.1.1, you should be familiar with what a pull request is.
Open GitHub desktop, and examine that the changes you’ve made are displayed in the Changes window:



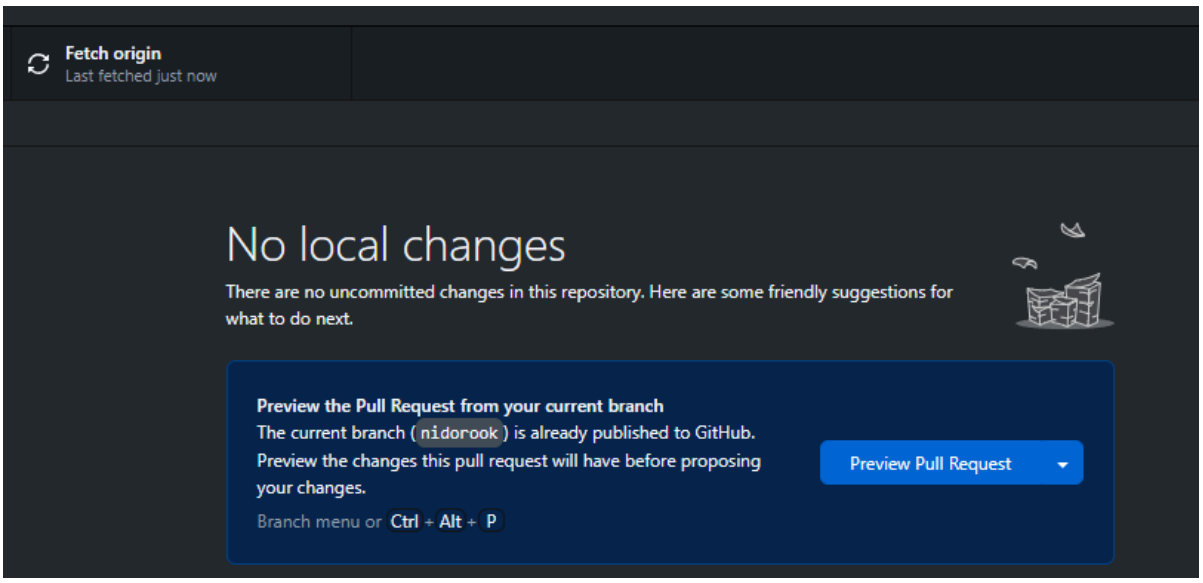
If you don't see your changes in the changes window, make sure you have selected Gravelmon as your current repository.

Next, create a new branch for your work. To do this, go to Branch > new branch at the top left corner of the app, and give your branch an appropriate name. If prompted, select "bring my changes to…".

Next, give your commit a title and a description. In our case, this will be the as follows:



Click on commit, and then on the push button in the top center of the app. It should have a '1' next to it, or however many commits youve made before pushing. Github desktop should give you the option to create a pull request for the changes.



Click on the Preview Pull Request button. This will open a window previewing your pull request. Click create pull request, which will open a window in your browser. If prompted to log in, do so with the account you used on github desktop. Next click on the bright green button named ‘create pull request’.

That’s all! Thank you for reading the guide! If you want to get faster or more efficient at integrating, feel free to read Chapter 7.

7. Tips and Tricks

- To work efficiently, it’s recommended to integrate more than 1 pokemon at a time. This allows you to go through each step for each pokemon as you reach it, minimising the amount of switching of tabs you’ll have to do. Take care to still thoroughly check each model you integrate.
- If you want to test a change without restarting your client in the dev environment, you can rebuild and then hit F3+T in game. This will reload the resources from the new build.